Section 8.6: Multi-Tenancy Patterns

> ***Module 8 of 8 · Isolation Models, Data Separation & Noisy Neighbor Mitigation***

Executive Overview

Multi-tenancy enables a single system to serve many customers cost-effectively while maintaining security, performance, and data isolation. The core tension is between isolation (strong security, compliance) and efficiency (shared resources, low cost). Getting this wrong causes data leaks, performance degradation for paying customers, or prohibitive infrastructure costs. This section covers isolation models, data separation strategies, and resource governance — with worked design problems.

Part 1: Tenant Isolation Models

1.1 Three Models Compared

Dimension	Silo	Pool	Bridge
Infrastructure	One stack per tenant	Shared stack for all	Hybrid by tier
Data separation	Separate DB per tenant	Row-level or schema per tenant	Separate for premium, shared for free
Isolation	Excellent	Fair	Variable
Cost per tenant	High	Low	Medium
Noisy neighbor risk	None	High	Low for premium
Compliance (SOC2, HIPAA)	Simple (audit one tenant at a time)	Complex (shared audit scope)	Medium
Onboarding new tenant	Slow (provision stack)	Fast (insert row)	Depends on tier
Number of tenants	Best for < 1,000	Best for > 10,000	Best for 1,000–100,000

1.2 Silo Model

```
Tenant A: [Load Balancer A] → [App Servers A] → [DB A]
Tenant B: [Load Balancer B] → [App Servers B] → [DB B]
Tenant C: [Load Balancer C] → [App Servers C] → [DB C]

Control Plane (shared): [Billing] [Provisioning] [Analytics]
```

When to use silo: - Enterprise contracts with dedicated infrastructure requirements - Compliance mandates physical separation (HIPAA BAA, FedRAMP) - Tenants with wildly different scale requirements - Tenants who need to customise schemas or application versions

When NOT to use silo: - > 10,000 tenants (operational overhead becomes unmanageable) - SMB/free tier users (per-tenant cost exceeds revenue)

1.3 Pool Model

```
All Tenants → [Shared Load Balancer] → [Shared App Cluster]
                                              ↓
                                [Tenant Middleware] extracts tenant_id
                                              ↓
                                [Shared Database] with tenant_id column
```

When to use pool: - High tenant count (10K–10M) - Homogeneous workloads per tenant - Free/low tier where per-tenant cost must be near zero - Tenants have no compliance requirements for physical separation

1.4 Bridge Model (Recommended for Most SaaS)

```
Free/Starter tenants → Pool (shared resources)
Premium tenants      → Silo (dedicated resources)
Enterprise tenants   → Silo + custom domain + dedicated DB

Control plane always shared:
  - Single admin dashboard
  - Unified billing API
  - Cross-tenant analytics
```

Part 2: Tenant Data Isolation Strategies

2.1 Four Strategies Compared

Strategy	Isolation	Storage Overhead	Migration Complexity	Cross-tenant Query	Max Tenants
Row-Level (tenant_id column)	Logical	None	Easy	Yes	Millions
Schema-per-tenant	Moderate	Low	Medium	Hard	~10,000
Database-per-tenant	Strong	High	Complex	Very hard	~1,000
Instance-per-tenant	Complete	Very high	Very complex	No	~100

2.2 Row-Level Security (Recommended for Pool Model)

```
-- Single table, all tenants
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    customer_name TEXT NOT NULL,
    total_cents BIGINT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Critical: composite index for tenant isolation at DB level
CREATE INDEX idx_orders_tenant_created ON orders (tenant_id, created_at DESC);

-- PostgreSQL Row Level Security — enforced at DB engine level
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON orders
    AS PERMISSIVE FOR ALL TO app_user
    USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
# Application sets tenant context before any query
def get_db_session(tenant_id: str):
    session = db.Session()
    session.execute(
        text("SET LOCAL app.current_tenant = :tid"),
        {"tid": tenant_id}
    )
```

```
    return session

# All queries through this session automatically filtered by RLS
# Even if application code forgets WHERE tenant_id = ?
def get_orders(tenant_id: str):
    with get_db_session(tenant_id) as session:
        return session.query(Order).all()  # RLS applies automatically
```

Why RLS matters: Application-level filtering (always adding `WHERE tenant_id = ?`) can be bypassed by a bug. RLS is enforced by PostgreSQL itself — even a SQL injection can only access the current tenant's rows.

2.3 Schema-Per-Tenant

```
-- Tenant provisioning creates a new schema
CREATE SCHEMA tenant_acme;
CREATE TABLE tenant_acme.orders (id UUID PRIMARY KEY, ...);
GRANT ALL ON SCHEMA tenant_acme TO tenant_acme_user;

-- Application connects with tenant-specific search_path
SET search_path TO tenant_acme;
SELECT * FROM orders;  -- Resolves to tenant_acme.orders
```

When schema-per-tenant is better than row-level: - Tenants need different schema versions (schema evolution per tenant) - Per-tenant backup/restore required without affecting others - Regulatory requirement to physically separate data within same DB instance - Expected tenant count < 5,000 (beyond this, 5,000 schemas become hard to manage)

2.4 Tenant-Aware Connection Routing

```
class TenantAwareConnectionPool:
    def __init__(self):
        # Free tenants share a pool (rate-limited)
        self.free_pool = ConnectionPool(min_size=10, max_size=200)
        # Premium tenants get a larger dedicated pool
        self.premium_pool = ConnectionPool(min_size=20, max_size=500)
        # Enterprise tenants get isolated pools keyed by tenant_id
        self.enterprise_pools: dict[str, ConnectionPool] = {}

    def get_connection(self, tenant_id: str) -> Connection:
        tenant = tenant_cache.get(tenant_id)

        if tenant.plan == 'enterprise':
            # Lazy-create dedicated pool for enterprise tenant
            if tenant_id not in self.enterprise_pools:
                self.enterprise_pools[tenant_id] = ConnectionPool(
                    dsn=tenant.dedicated_db_url, min_size=5, max_size=50
                )
```

```
            return self.enterprise_pools[tenant_id].acquire()

        elif tenant.plan == 'premium':
            return self.premium_pool.acquire()

        else:
            # Free tier: check quota before acquiring
            if self.free_pool.available_connections() < 10:
                raise TooManyRequestsError(tenant_id)
            return self.free_pool.acquire()
```

Part 3: Noisy Neighbor Mitigation

3.1 Rate Limiting

```
from redis import Redis
import time

class TenantRateLimiter:
    """Token bucket rate limiter per tenant."""

    def __init__(self, redis: Redis):
        self.redis = redis

    def check_and_consume(self, tenant_id: str, plan: str) -> bool:
        limits = {
            'free': 60,       # 60 requests/minute
            'starter': 300,
            'premium': 3000,
            'enterprise': 30000
        }
        limit = limits.get(plan, 60)
        key = f"rate:{tenant_id}:{int(time.time() // 60)}"  # Per-minute window

        pipe = self.redis.pipeline()
        pipe.incr(key)
        pipe.expire(key, 120)  # 2-minute TTL
        results = pipe.execute()

        current = results[0]
        if current > limit:
            return False  # Rate limit exceeded
        return True

# Middleware
def rate_limit_middleware(request):
    tenant_id = request.headers.get('X-Tenant-ID')
    plan = get_tenant_plan(tenant_id)
```

```
    if not rate_limiter.check_and_consume(tenant_id, plan):
        return Response(status=429, headers={
            'Retry-After': '60',
            'X-RateLimit-Plan': plan
        })
```

3.2 Kubernetes Resource Quotas Per Tenant Namespace

```
# Each tenant gets a Kubernetes namespace
apiVersion: v1
kind: Namespace
metadata:
  name: tenant-acme
  labels: { tenant: acme, plan: premium }
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-quota
  namespace: tenant-acme
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
    services: "5"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: tenant-limits
  namespace: tenant-acme
spec:
  limits:
  - type: Container
    default: { cpu: 500m, memory: 512Mi }       # Default limit
    defaultRequest: { cpu: 100m, memory: 128Mi } # Default request
    max: { cpu: "2", memory: 4Gi }              # Per-container cap
```

3.3 Priority Queue for Request Processing

```
import heapq
from dataclasses import dataclass, field
from enum import IntEnum

class Priority(IntEnum):
    ENTERPRISE = 1  # Highest priority
    PREMIUM    = 2
```

```
    STARTER    = 3
    FREE       = 4  # Lowest priority

@dataclass(order=True)
class Request:
    priority: Priority
    timestamp: float
    tenant_id: str = field(compare=False)
    payload: dict = field(compare=False)

class PriorityRequestQueue:
    def __init__(self):
        self._queue = []
        self._lock = threading.Lock()

    def enqueue(self, tenant_id: str, payload: dict):
        plan = get_tenant_plan(tenant_id)
        priority = Priority[plan.upper()]
        req = Request(priority, time.time(), tenant_id, payload)
        with self._lock:
            heapq.heappush(self._queue, req)

    def dequeue(self) -> Request:
        with self._lock:
            return heapq.heappop(self._queue)
```

Part 4: Interview Design Problems

Problem 1 — 100K Tenants, 1% Premium: Database Connection Management

Question: Design a multi-tenant SaaS platform with 100K tenants where 1% are premium with dedicated resources. How do you prevent free tenants from consuming all database connections?

Solution:

```
# Architecture: Three-tier connection governance

class MultiTenantConnectionGovernor:
    TOTAL_DB_CONNECTIONS = 2000  # max_connections in PostgreSQL

    # Allocation:
    #   - 1,000 premium tenants × 1.5 connections avg = 1,500 (dedicated pool)
    #   - 99,000 free tenants × share of 400 connections (shared pool)
    #   - 100 reserved for admin/migrations
    PREMIUM_POOL_MAX = 1500
    FREE_POOL_MAX    = 400
    ADMIN_RESERVED   = 100

    def __init__(self):
```

```
        self.premium_pool = ConnectionPool(max_size=self.PREMIUM_POOL_MAX)
        self.free_pool    = ConnectionPool(max_size=self.FREE_POOL_MAX)

    def acquire(self, tenant_id: str) -> Connection:
        plan = self._get_plan_cached(tenant_id)

        if plan == 'premium':
            # Never wait more than 100ms
            conn = self.premium_pool.acquire(timeout_ms=100)
            if conn is None:
                # Extremely rare: 1,500 premium connections all active
                raise
ServiceUnavailableError("Premium capacity exceeded — contact support")
            return conn
        else:
            # Free tier: refuse if pool is stressed
            utilisation = self.free_pool.active_count() / self.FREE_POOL_MAX
            if utilisation > 0.95:
                # Let 5% of free requests through during overload (fairness)
                if random.random() > 0.05:
                    raise TooManyRequestsError(tenant_id)

            conn = self.free_pool.acquire(timeout_ms=500)
            if conn is None:
                raise TooManyRequestsError(tenant_id)
            return conn

# PostgreSQL-level enforcement (second line of defense)
# ALTER ROLE free_tenant_user CONNECTION LIMIT 400;
# ALTER ROLE premium_tenant_user CONNECTION LIMIT 1500;
```

Statistical justification for the free pool:

```
99,000 free tenants, 400 connection budget
At any given second, active free tenants = 99,000 × 0.001 (0.1% concurrency rate)
= 99 active tenants simultaneously
99 tenants × avg 2 concurrent queries = 198 connections needed
400 budget = 2× headroom — works fine under normal load

Under spike: rate limiting (60 req/min/tenant) prevents runaway tenants
Emergency circuit breaker: if free_pool > 95%, shed load proactively
```

Problem 2 — Cross-Tenant Data Leakage Prevention

Question: A bug in your application code omits the `WHERE tenant_id = ?` clause on a query. What layers of defence prevent cross-tenant data exposure?

Solution: Defence in Depth

```
Layer 1 — Application: ORM/query builder enforces tenant_id
  class TenantScopedRepository:
      def find(self, id):
          # tenant_id injected automatically — developer cannot forget
          return db.query(Model).filter(
              Model.id == id,
              Model.tenant_id == g.tenant_id  # from request context
          ).first()

Layer 2 — Database: Row Level Security (PostgreSQL)
  Even if application sends: SELECT * FROM orders WHERE id = 42
  PostgreSQL rewrites to: SELECT * FROM orders WHERE id = 42 AND tenant_id =
'<current>'
  Bug in app code cannot bypass RLS

Layer 3 — Database user permissions
  Each tenant class gets a separate DB user
  free_user can only access tables it owns; cannot cross schemas

Layer 4 — Connection-level context
  Every connection sets: SET LOCAL app.current_tenant = '<id>'
  Connection returned to pool ONLY after rollback/commit clears context
  Prevents tenant context leak across connection reuse

Layer 5 — Audit logging
  Every query > threshold logged with tenant_id
  Anomaly detection: tenant A querying orders not in its RLS domain
  → Alert and automatic connection termination
```

Problem 3 — Tenant Onboarding at Scale

Question: You need to onboard 1,000 new tenants per day automatically. How do you provision tenant resources without manual steps or schema migration downtime?

Solution:

```
# Automated tenant provisioning service
class TenantProvisioningService:

    def provision(self, tenant_request: TenantRequest) -> Tenant:
        tenant_id = uuid4()

        # Step 1: Create tenant record (immediate; no schema change)
        tenant = Tenant(
            id=tenant_id,
            name=tenant_request.company_name,
            plan=tenant_request.plan,
            status='PROVISIONING'
        )
```

```
        tenant_repo.save(tenant)

        # Step 2: Plan-specific provisioning (async, ~30s)
        if tenant_request.plan == 'enterprise':
            self._provision_enterprise(tenant)
        elif tenant_request.plan == 'premium':
            self._provision_premium_schema(tenant)
        else:
            # Free tier: row-level isolation; zero provisioning needed
            self._activate_free_tenant(tenant)

        return tenant

    def _provision_premium_schema(self, tenant: Tenant):
        # Schema provisioning via migration tool (Flyway/Liquibase)
        # Each schema is a copy of the "template" schema
        schema_name = f"t_{tenant.id.hex}"

        with admin_db.connect() as conn:
            conn.execute(text(f"CREATE SCHEMA {schema_name}"))
            # Apply migrations to new schema only
            flyway.migrate(schema=schema_name, locations="db/tenant-migrations")
            conn.execute(text(
                f"GRANT ALL ON SCHEMA {schema_name} TO {schema_name}_user"
            ))

        tenant.schema_name = schema_name
        tenant.status = 'ACTIVE'
        tenant_repo.save(tenant)

    def _provision_enterprise(self, tenant: Tenant):
        # Terraform/Pulumi creates dedicated RDS instance
        infra = infra_provisioner.create_rds_instance(
            tenant_id=str(tenant.id),
            instance_class='db.t3.medium',
            tags={'tenant': str(tenant.id), 'plan': 'enterprise'}
        )
        # Wait for DB ready (usually 5-8 min)
        infra.wait_until_available()
        tenant.db_endpoint = infra.endpoint
        tenant.status = 'ACTIVE'
        tenant_repo.save(tenant)
```

Quick Reference

Model	Best Tenant Count	Isolation	Use For
Silo	< 1,000	Complete	Enterprise, compliance-heavy
Pool	> 10,000	Logical (RLS)	SMB, free tier, startups

Model	Best Tenant Count	Isolation	Use For
Bridge	1,000–100,000	Tiered	Most SaaS products

Data Strategy	Max Tenants	When to Choose
Row-level (tenant_id)	Millions	Default; simplest to operate
Schema-per-tenant	~10,000	Per-tenant backups; schema variation
DB-per-tenant	~1,000	Enterprise compliance; strong isolation